

DAY 6- LAB PROGRAMMING

CSA0617-Design and Analysis of Algorithms

1. Bellman-Ford Algorithm

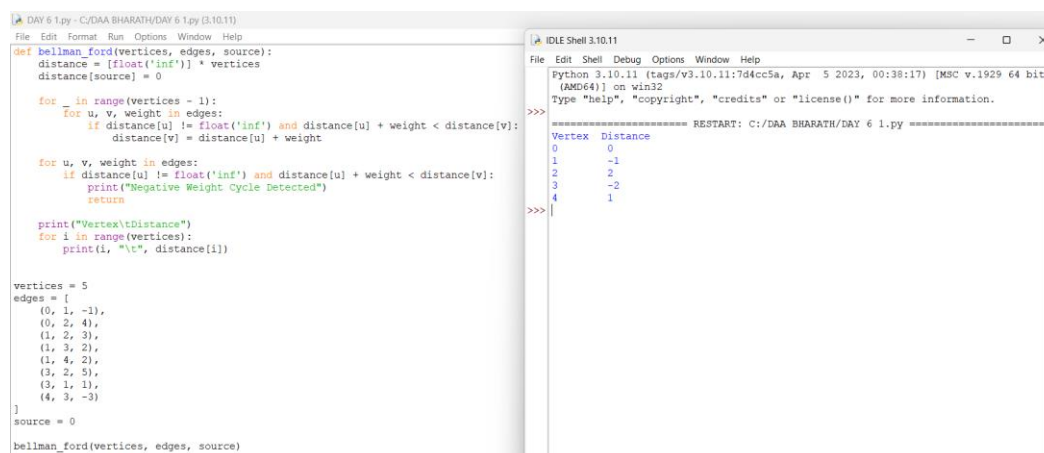
Aim

To implement the Bellman-Ford Algorithm to find the shortest distance from a given source vertex to all other vertices in a weighted directed graph and detect the presence of a negative weight cycle.

Algorithm

1. Start the program.
2. Initialize the distance of all vertices to infinity and set the source vertex distance to 0.
3. Repeat the following steps $V - 1$ times, where V is the number of vertices:
 - For every edge (u, v, weight) , check whether $\text{distance}[u] + \text{weight} < \text{distance}[v]$.
 - If true, update $\text{distance}[v] = \text{distance}[u] + \text{weight}$.
4. After $V - 1$ iterations, check all edges once again.
5. If any distance can still be reduced, a negative weight cycle exists.
6. Otherwise, display the shortest distance from the source vertex to every vertex.
7. Stop the program

PROGRAM & OUTPUT



```
DAY 6 1.py - C:/DAA BHARATH/DAY 6 1.py (3.10.11)
File Edit Format Run Options Window Help
def bellman_ford(vertices, edges, source):
    distance = [float('inf')] * vertices
    distance[source] = 0

    for _ in range(vertices - 1):
        for u, v, weight in edges:
            if distance[u] != float('inf') and distance[u] + weight < distance[v]:
                distance[v] = distance[u] + weight

    for u, v, weight in edges:
        if distance[u] != float('inf') and distance[u] + weight < distance[v]:
            print("Negative Weight Cycle Detected")
            return

    print("Vertex\tDistance")
    for i in range(vertices):
        print(i, "\t", distance[i])

vertices = 5
edges = [
    (0, 1, -1),
    (0, 2, 4),
    (1, 2, 3),
    (1, 3, 2),
    (1, 4, 2),
    (3, 2, 5),
    (3, 1, 1),
    (4, 3, -3)
]
source = 0
bellman_ford(vertices, edges, source)
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 1.py =====
Vertex Distance
0      0
1     -1
2      2
3     -2
4      1
>>>
```

2. Floyd-Warshall Algorithm

Aim

To implement the Floyd-Warshall Algorithm to find the shortest distances between every pair of vertices in a weighted graph represented using an adjacency matrix.

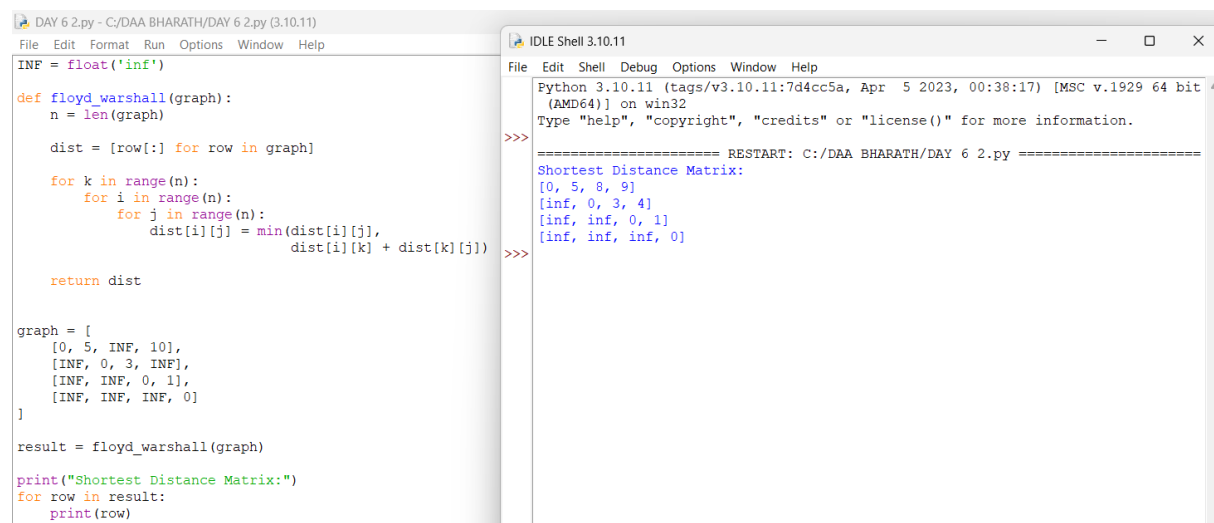
Algorithm

1. Start the program.
2. Initialize the given adjacency matrix as the distance matrix.
3. Consider each vertex as an intermediate vertex one by one.
4. For every pair of vertices i and j , check whether the path through the intermediate vertex k is shorter.
5. Update the distance using:
$$\text{distance}[i][j] = \min(\text{distance}[i][j], \text{distance}[i][k] + \text{distance}[k][j]).$$
6. Repeat the process for all vertices.
7. Display the final matrix containing the shortest distances between every pair of vertices.

Time Complexity: $O(V^3)$

Space Complexity: $O(V^2)$

PROGRAM & OUTPUT



```
DAY 6 2.py - C:/DAA BHARATH/DAY 6 2.py (3.10.11)
File Edit Format Run Options Window Help

INF = float('inf')

def floyd_warshall(graph):
    n = len(graph)

    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j],
                                dist[i][k] + dist[k][j])

    return dist

graph = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]

result = floyd_warshall(graph)

print("Shortest Distance Matrix:")
for row in result:
    print(row)
```

```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/DAY 6 2.py =====
Shortest Distance Matrix:
[0, 5, 8, 9]
[inf, 0, 3, 4]
[inf, inf, 0, 1]
[inf, inf, inf, 0]
>>>
```

3. Warshall's Algorithm

Aim

To implement Warshall's Algorithm to find the transitive closure of a directed graph and determine whether a path exists between every pair of vertices.

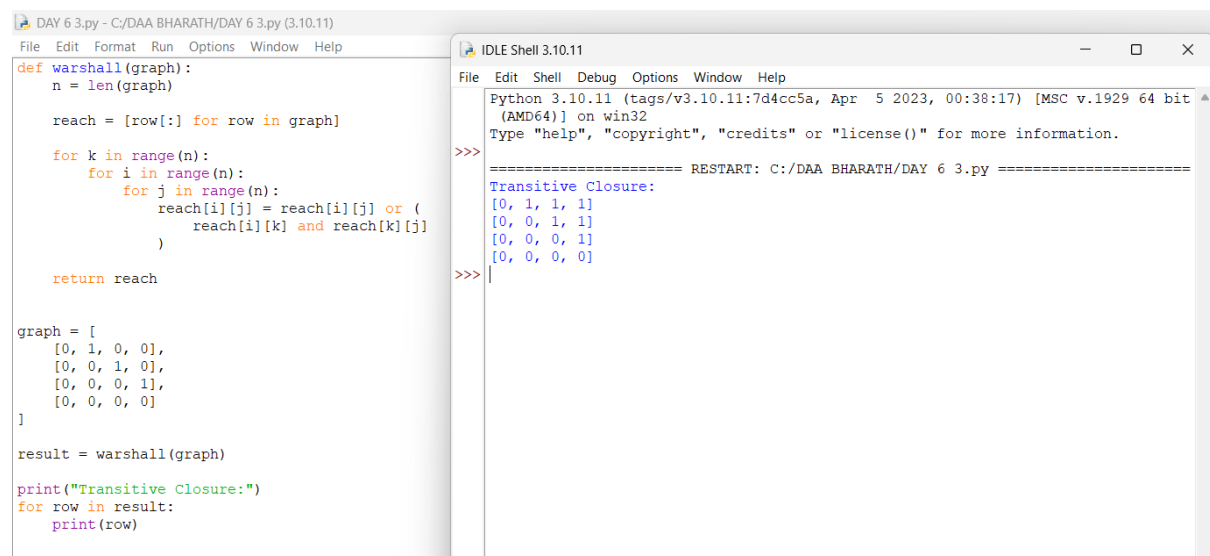
Algorithm

1. Start the program.
2. Initialize the given adjacency matrix as the reachability matrix.
3. Consider each vertex as an intermediate vertex one by one.
4. For every pair of vertices i and j , check whether vertex j is reachable from vertex i through vertex k .
5. Update the matrix using:
$$\text{reach}[i][j] = \text{reach}[i][j] \text{ OR } (\text{reach}[i][k] \text{ AND } \text{reach}[k][j]).$$
6. Repeat the process for all vertices.
7. Display the final matrix as the transitive closure of the graph.

Time Complexity: $O(V^3)$

Space Complexity: $O(V^2)$

PROGRAM & OUTPUT



The screenshot displays a Python IDE with two windows. The left window, titled 'DAY 6 3.py - C:/DAA BHARATH/DAY 6 3.py (3.10.11)', contains the following code:

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )

    return reach

graph = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

result = warshall(graph)
print("Transitive Closure:")
for row in result:
    print(row)
```

The right window, titled 'IDLE Shell 3.10.11', shows the output of the program:

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/DAA BHARATH/DAY 6 3.py =====
Transitive Closure:
[0, 1, 1, 1]
[0, 0, 1, 1]
[0, 0, 0, 1]
[0, 0, 0, 0]
>>>
```

4. Find the City with the Smallest Number of Reachable Cities

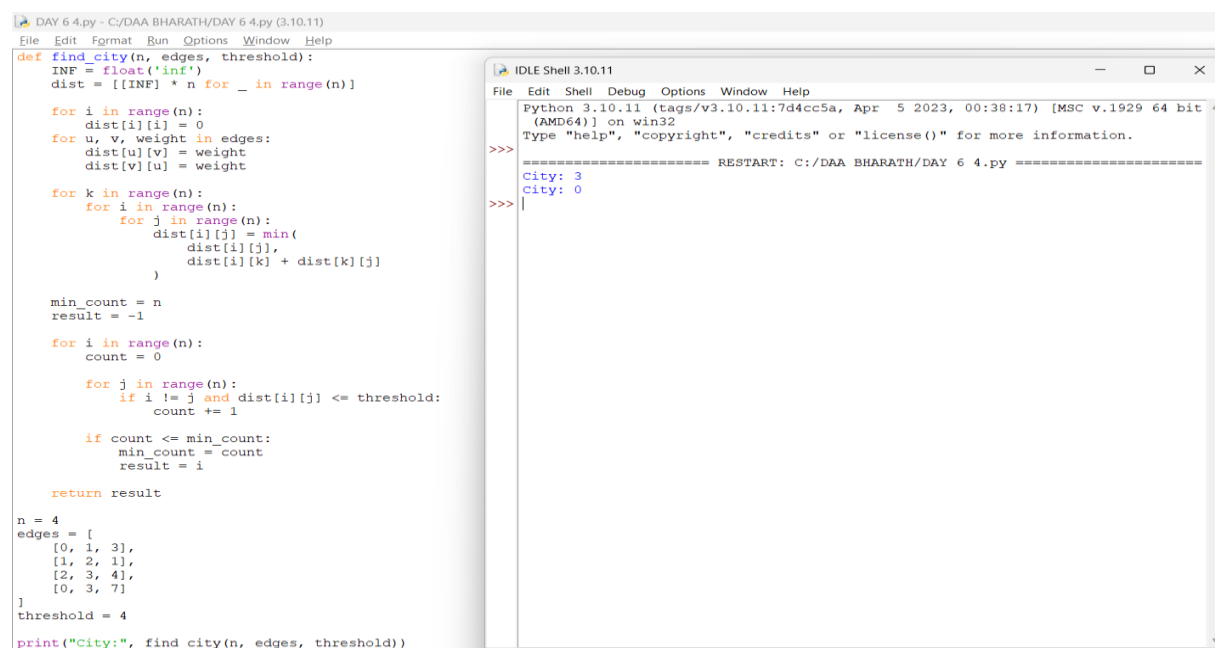
Aim

To find the city that can reach the smallest number of other cities within a given distance threshold using the Floyd-Warshall Algorithm.

Algorithm

1. Start the program.
2. Initialize the distance matrix with infinity and set the distance from each city to itself as 0.
3. Store the given edge weights in the distance matrix.
4. Apply the Floyd-Warshall algorithm to find the shortest distance between every pair of cities.
5. For each city, count the number of other cities whose shortest distance is less than or equal to the given threshold.
6. Select the city with the smallest number of reachable cities.
7. If there is a tie, select the city with the greater city number.
8. Display the selected city.

PROGRAM & OUTPUT



```
DAY 6 4.py - C:/DAA BHARATH/DAY 6 4.py (3.10.11)
File Edit Format Run Options Window Help
def find_city(n, edges, threshold):
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0
    for u, v, weight in edges:
        dist[u][v] = weight
        dist[v][u] = weight

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    min_count = n
    result = -1

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1

        if count <= min_count:
            min_count = count
            result = i

    return result

n = 4
edges = [
    [0, 1, 3],
    [1, 2, 1],
    [2, 3, 4],
    [0, 3, 7]
]
threshold = 4
print("City:", find_city(n, edges, threshold))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 4.py =====
City: 3
City: 0
>>>
```

5. Network Delay Time

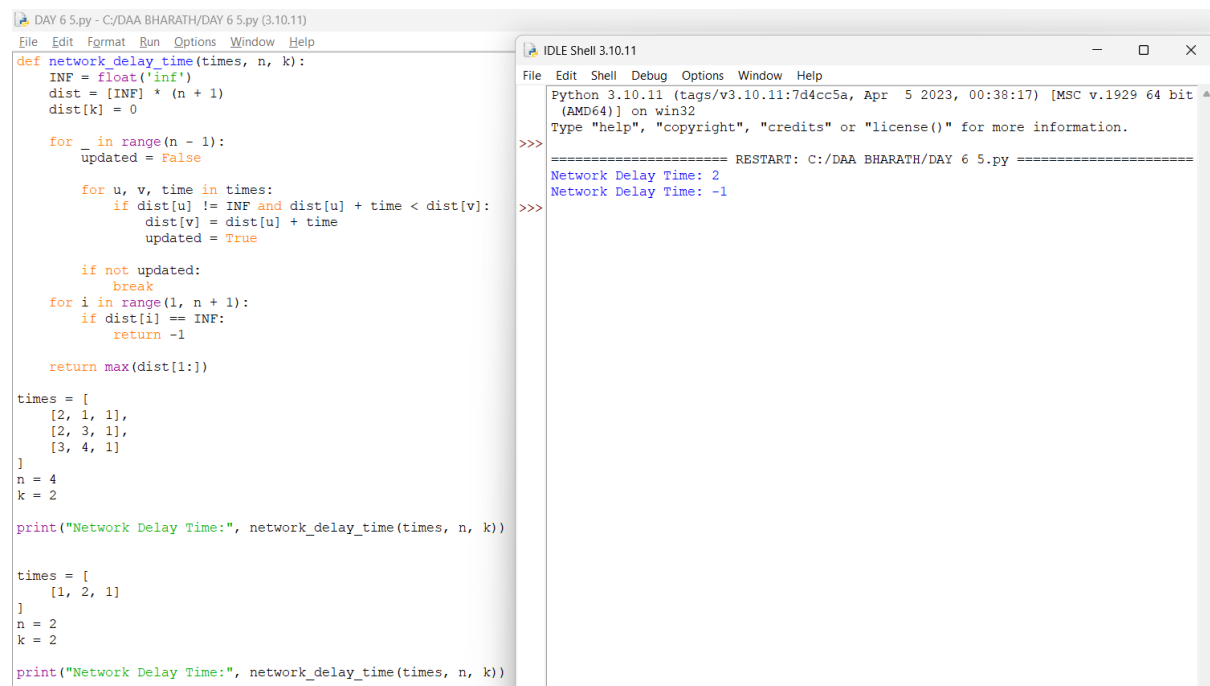
Aim

To find the minimum time required for a signal to reach all nodes in a network using the Bellman-Ford Algorithm, and return -1 if some nodes cannot be reached.

Algorithm

1. Start the program.
2. Initialize the distance of all nodes to infinity.
3. Set the distance of the source node k to 0.
4. Relax all edges $n - 1$ times using the Bellman-Ford algorithm.
5. Find the maximum shortest distance among all nodes.
6. If any node still has an infinite distance, return -1.
7. Otherwise, return the maximum distance as the network delay time.
8. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 5.py - C:/DAA BHARATH/DAY 6 5.py (3.10.11)
File Edit Format Run Options Window Help
def network_delay_time(times, n, k):
    INF = float('inf')
    dist = [INF] * (n + 1)
    dist[k] = 0

    for _ in range(n - 1):
        updated = False

        for u, v, time in times:
            if dist[u] != INF and dist[u] + time < dist[v]:
                dist[v] = dist[u] + time
                updated = True

        if not updated:
            break
    for i in range(1, n + 1):
        if dist[i] == INF:
            return -1

    return max(dist[1:])

times = [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]
n = 4
k = 2

print("Network Delay Time:", network_delay_time(times, n, k))

times = [
    [1, 2, 1]
]
n = 2
k = 2

print("Network Delay Time:", network_delay_time(times, n, k))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 5.py =====
Network Delay Time: 2
Network Delay Time: -1
>>>
```

6. Cheapest Flight Within K Stops

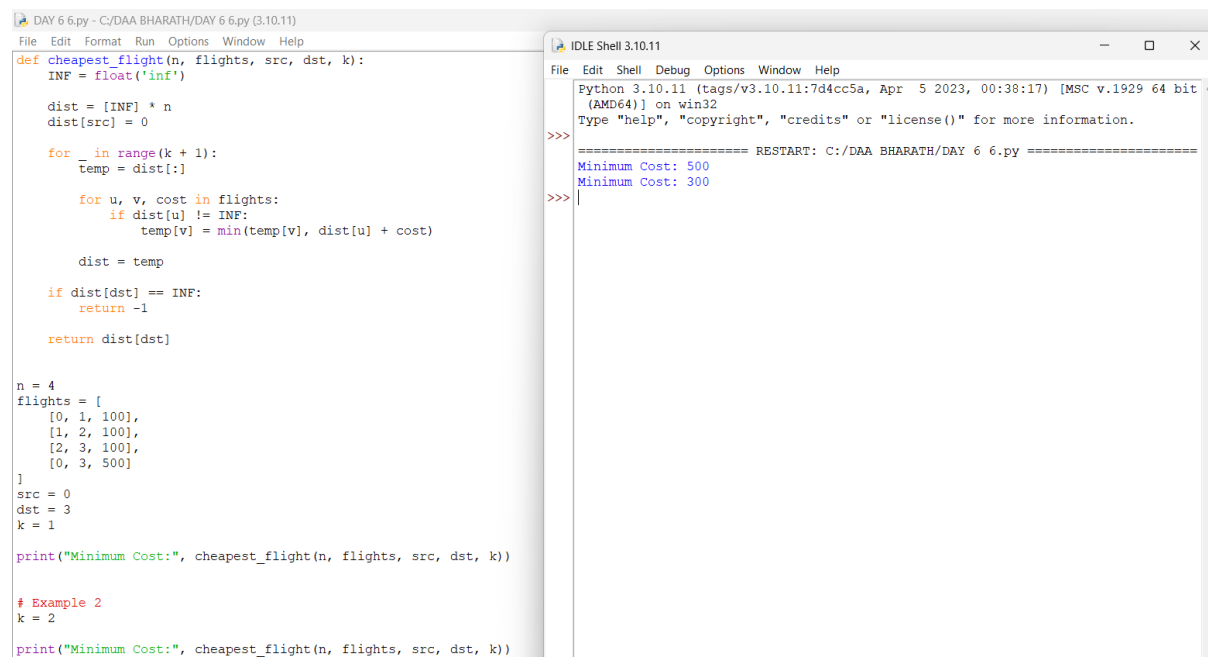
Aim

To find the minimum cost flight from a source city to a destination city with at most K stops using the Bellman-Ford Algorithm.

Algorithm

1. Start the program.
2. Initialize the distance of all cities to infinity.
3. Set the distance of the source city to 0.
4. Relax all flights for $K + 1$ iterations.
5. Use a temporary copy of the distance array during each iteration to avoid using more than the allowed number of stops.
6. Update the cost if a cheaper route is found.
7. After $K + 1$ iterations, check the distance of the destination city.
8. Display the minimum cost. If the destination cannot be reached, display -1.

PROGRAM & OUTPUT



```
DAY 6 6.py - C:/DAA BHARATH/DAY 6 6.py (3.10.11)
File Edit Format Run Options Window Help
def cheapest_flight(n, flights, src, dst, k):
    INF = float('inf')

    dist = [INF] * n
    dist[src] = 0

    for _ in range(k + 1):
        temp = dist[:]

        for u, v, cost in flights:
            if dist[u] != INF:
                temp[v] = min(temp[v], dist[u] + cost)

        dist = temp

    if dist[dst] == INF:
        return -1

    return dist[dst]

n = 4
flights = [
    [0, 1, 100],
    [1, 2, 100],
    [2, 3, 100],
    [0, 3, 500]
]
src = 0
dst = 3
k = 1

print("Minimum Cost:", cheapest_flight(n, flights, src, dst, k))

# Example 2
k = 2
print("Minimum Cost:", cheapest_flight(n, flights, src, dst, k))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 6.py =====
Minimum Cost: 500
Minimum Cost: 300
>>>
```

7. Detect Negative Weight Cycle

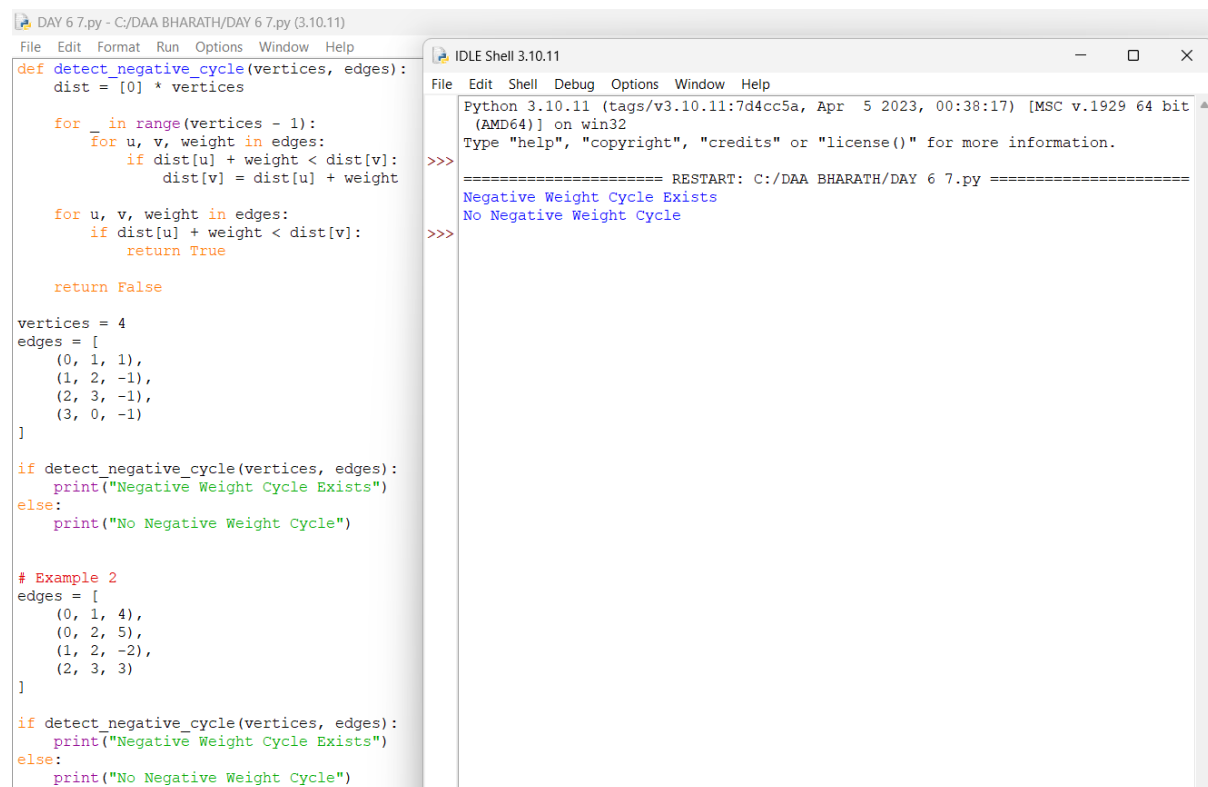
Aim

To implement the Bellman-Ford Algorithm to detect whether a weighted directed graph contains a negative weight cycle.

Algorithm

1. Start the program.
2. Initialize the distance of all vertices to 0.
3. Relax all edges $V - 1$ times, where V is the number of vertices.
4. Perform one additional iteration over all edges.
5. If any distance can still be reduced, a negative weight cycle exists.
6. Otherwise, conclude that there is no negative weight cycle.
7. Display the result.
8. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 7.py - C:/DAA BHARATH/DAY 6 7.py (3.10.11)
File Edit Format Run Options Window Help
def detect_negative_cycle(vertices, edges):
    dist = [0] * vertices

    for _ in range(vertices - 1):
        for u, v, weight in edges:
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight

    for u, v, weight in edges:
        if dist[u] + weight < dist[v]:
            return True

    return False

vertices = 4
edges = [
    (0, 1, 1),
    (1, 2, -1),
    (2, 3, -1),
    (3, 0, -1)
]

if detect_negative_cycle(vertices, edges):
    print("Negative Weight Cycle Exists")
else:
    print("No Negative Weight Cycle")

# Example 2
edges = [
    (0, 1, 4),
    (0, 2, 5),
    (1, 2, -2),
    (2, 3, 3)
]

if detect_negative_cycle(vertices, edges):
    print("Negative Weight Cycle Exists")
else:
    print("No Negative Weight Cycle")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 7.py =====
Negative Weight Cycle Exists
No Negative Weight Cycle
>>>
```

8. Determine Reachability Between Two Vertices

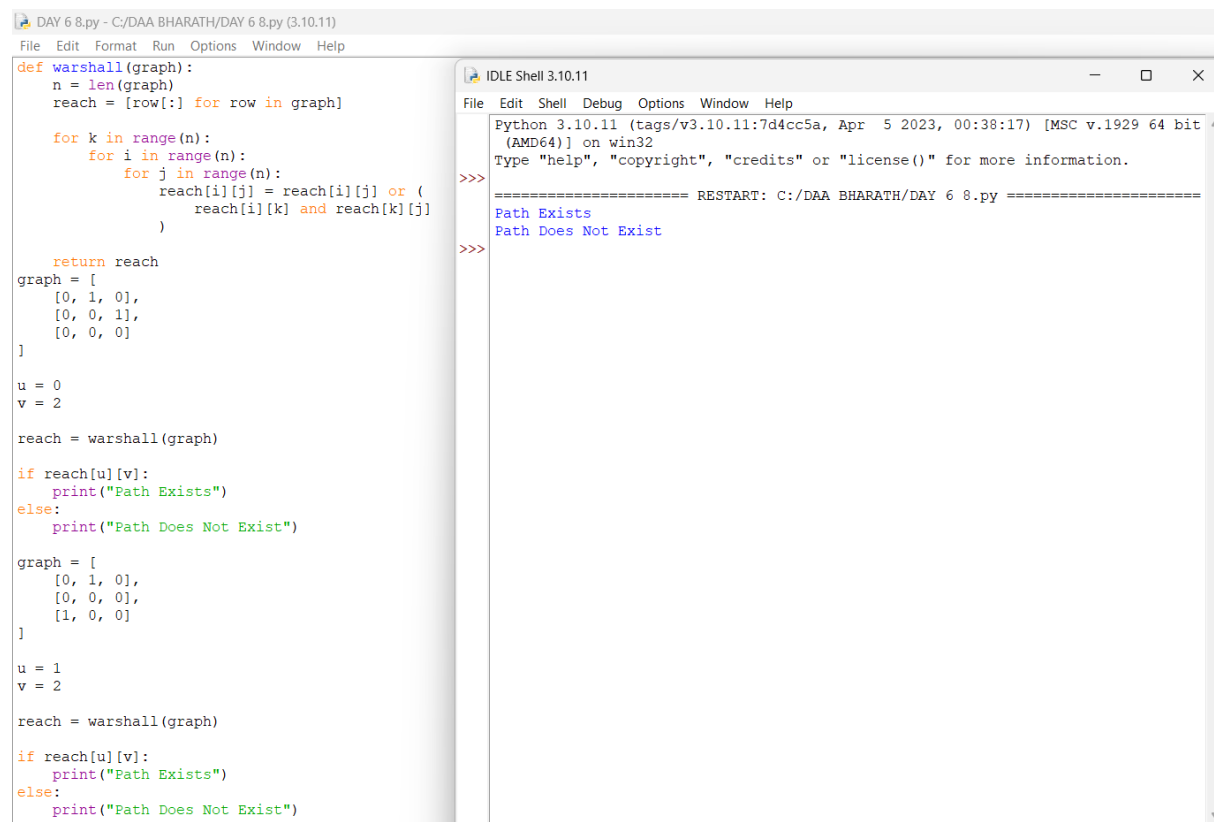
Aim

To determine whether a path exists between two given vertices in a directed graph using Warshall's Algorithm.

Algorithm

1. Start the program.
2. Read the adjacency matrix of the directed graph.
3. Initialize the reachability matrix with the given adjacency matrix.
4. Consider each vertex k as an intermediate vertex.
5. For every pair of vertices i and j , update:
 $\text{reach}[i][j] = \text{reach}[i][j] \text{ OR } (\text{reach}[i][k] \text{ AND } \text{reach}[k][j])$.
6. Repeat the process for all vertices.
7. Check $\text{reach}[u][v]$.
8. If it is 1, print Path Exists; otherwise, print Path Does Not Exist.

PROGRAM & OUTPUT



```
DAY 6 8.py - C:/DAA BHARATH/DAY 6 8.py (3.10.11)
File Edit Format Run Options Window Help
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )

    return reach

graph = [
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0]
]

u = 0
v = 2

reach = warshall(graph)

if reach[u][v]:
    print("Path Exists")
else:
    print("Path Does Not Exist")

graph = [
    [0, 1, 0],
    [0, 0, 0],
    [1, 0, 0]
]

u = 1
v = 2

reach = warshall(graph)

if reach[u][v]:
    print("Path Exists")
else:
    print("Path Does Not Exist")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 8.py =====
Path Exists
Path Does Not Exist
>>>
```

9. Count Reachable Vertices from a Source

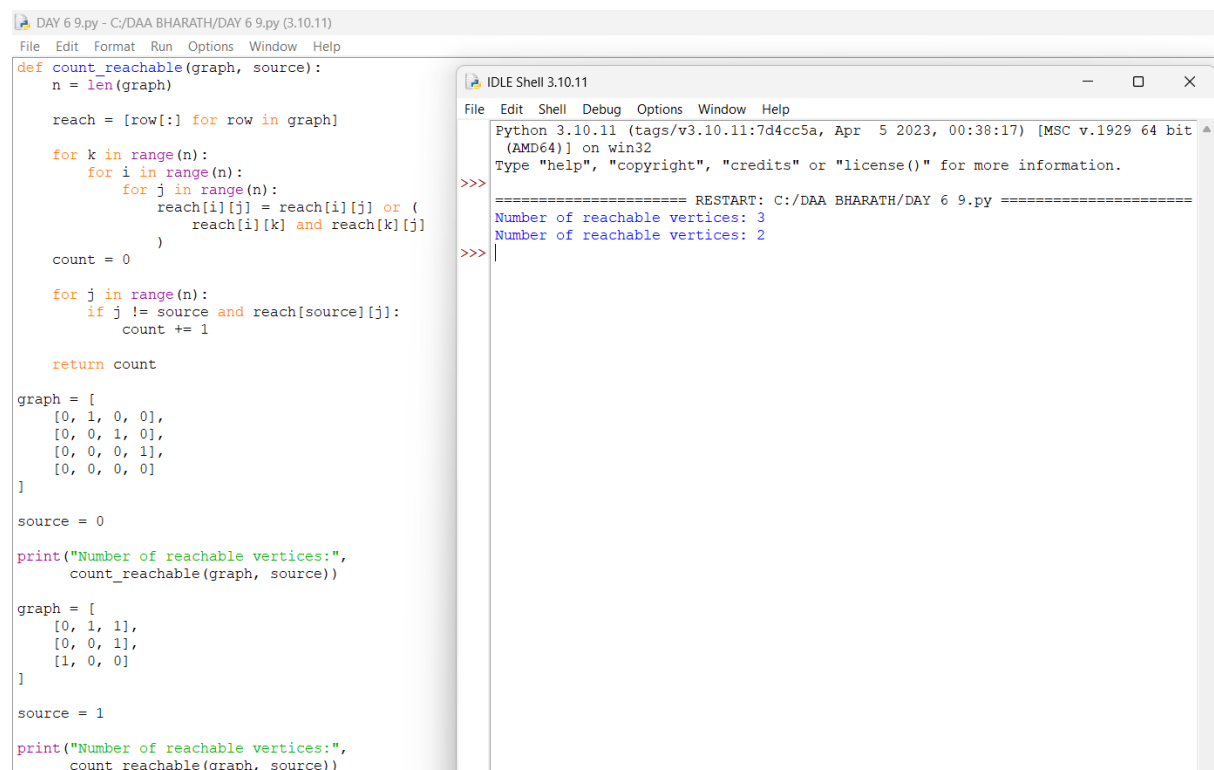
Aim

To find the number of vertices reachable from a given source vertex in a directed graph using the transitive closure generated by Warshall's Algorithm.

Algorithm

1. Start the program.
2. Read the adjacency matrix and source vertex.
3. Initialize the reachability matrix with the adjacency matrix.
4. Apply Warshall's Algorithm to calculate the transitive closure.
5. Check every vertex in the source row of the reachability matrix.
6. Count the vertices whose value is 1.
7. Do not count the source vertex itself.
8. Display the total number of reachable vertices.
9. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 9.py - C:/DAA BHARATH/DAY 6 9.py (3.10.11)
File Edit Format Run Options Window Help
def count_reachable(graph, source):
    n = len(graph)

    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )

    count = 0

    for j in range(n):
        if j != source and reach[source][j]:
            count += 1

    return count

graph = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

source = 0

print("Number of reachable vertices:",
      count_reachable(graph, source))

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

source = 1

print("Number of reachable vertices:",
      count_reachable(graph, source))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 9.py =====
Number of reachable vertices: 3
Number of reachable vertices: 2
>>>
```

10. Find the Diameter of a Graph

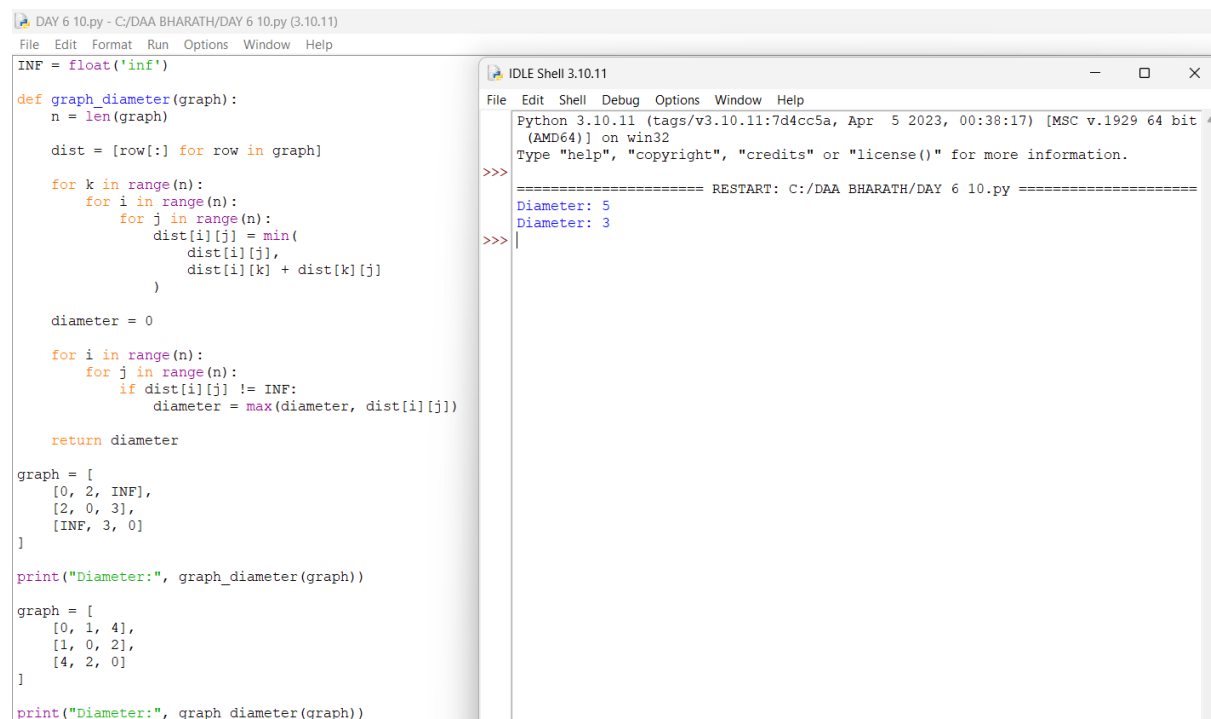
Aim

To find the diameter of a weighted graph, which is the maximum shortest-path distance between any pair of vertices, using the Floyd-Warshall Algorithm.

Algorithm

1. Start the program.
2. Initialize the distance matrix with the given graph.
3. Consider every vertex as an intermediate vertex.
4. Apply the Floyd-Warshall formula to find the shortest distance between every pair of vertices.
5. Find the maximum finite shortest-path distance in the resulting matrix.
6. Display this maximum distance as the diameter of the graph.
7. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 10.py - C:/DAA BHARATH/DAY 6 10.py (3.10.11)
File Edit Format Run Options Window Help
INF = float('inf')

def graph_diameter(graph):
    n = len(graph)

    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    diameter = 0

    for i in range(n):
        for j in range(n):
            if dist[i][j] != INF:
                diameter = max(diameter, dist[i][j])

    return diameter

graph = [
    [0, 2, INF],
    [2, 0, 3],
    [INF, 3, 0]
]

print("Diameter:", graph_diameter(graph))

graph = [
    [0, 1, 4],
    [1, 0, 2],
    [4, 2, 0]
]

print("Diameter:", graph_diameter(graph))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 10.py =====
Diameter: 5
Diameter: 3
>>>
```

11. Find the Shortest Distance Between Two Vertices

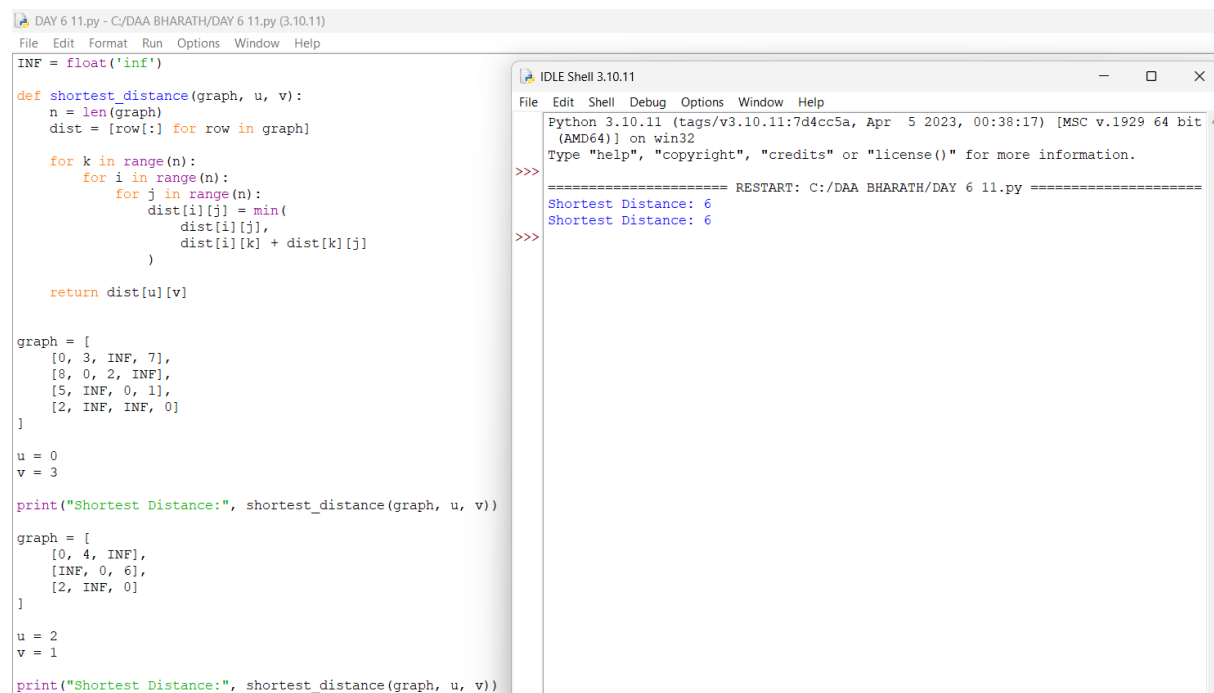
Aim

To find the shortest distance between two given vertices in a weighted graph using the Floyd-Warshall Algorithm.

Algorithm

1. Start the program.
2. Initialize the distance matrix with the given graph.
3. Consider each vertex as an intermediate vertex.
4. Update the shortest distance between every pair of vertices using Floyd-Warshall.
5. After completing the algorithm, access `dist[u][v]`.
6. Display the shortest distance between vertices `u` and `v`.
7. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 11.py - C:/DAA BHARATH/DAY 6 11.py (3.10.11)
File Edit Format Run Options Window Help

INF = float('inf')

def shortest_distance(graph, u, v):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    return dist[u][v]

graph = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

u = 0
v = 3

print("Shortest Distance:", shortest_distance(graph, u, v))

graph = [
    [0, 4, INF],
    [INF, 0, 6],
    [2, INF, 0]
]

u = 2
v = 1

print("Shortest Distance:", shortest_distance(graph, u, v))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> ===== RESTART: C:/DAA BHARATH/DAY 6 11.py =====
Shortest Distance: 6
Shortest Distance: 6
>>>
```

12. Find a Vertex Reachable from Every Other Vertex

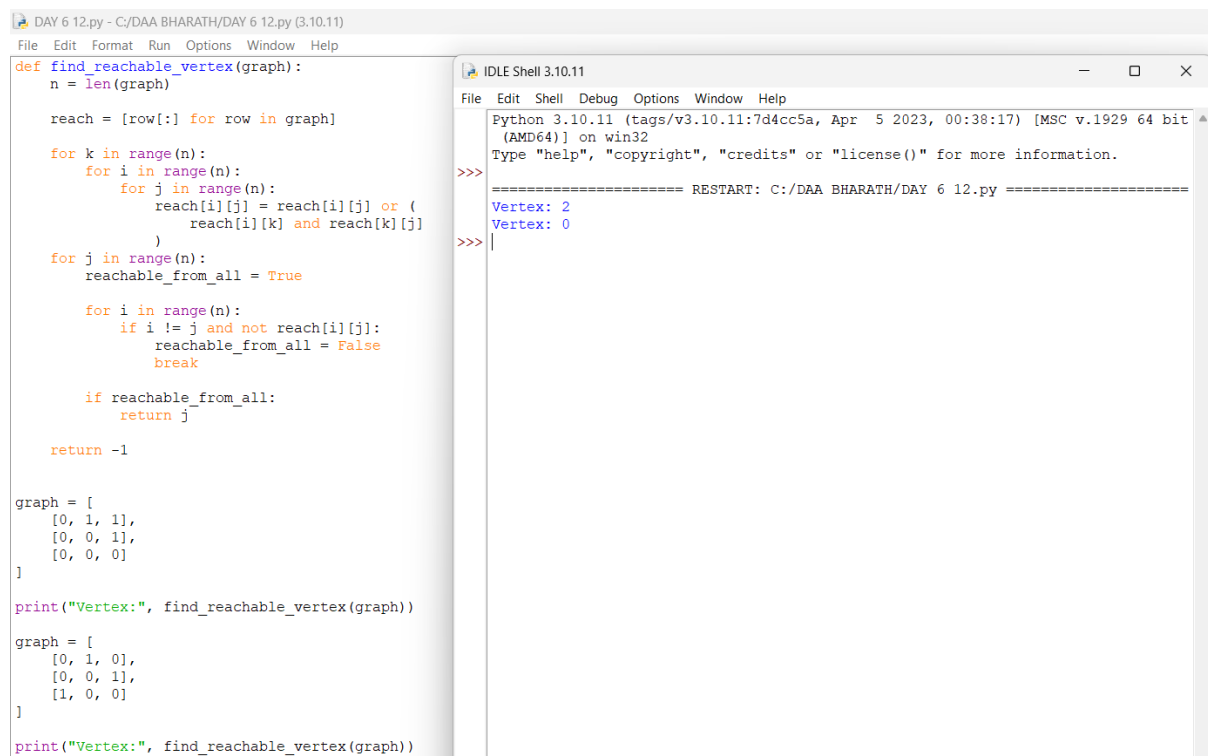
Aim

To find the smallest indexed vertex that is reachable from every other vertex in a directed graph using Warshall's Algorithm.

Algorithm

1. Start the program.
2. Initialize the reachability matrix using the given adjacency matrix.
3. Apply Warshall's Algorithm to calculate the transitive closure.
4. For each vertex j , check whether every other vertex can reach j .
5. If $\text{reach}[i][j]$ is true for all vertices i , select vertex j .
6. Since vertices are checked from smallest to largest index, return the smallest valid vertex.
7. If no such vertex exists, display -1.
8. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 12.py - C:/DAA BHARATH/DAY 6 12.py (3.10.11)
File Edit Format Run Options Window Help

def find_reachable_vertex(graph):
    n = len(graph)

    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )
    for j in range(n):
        reachable_from_all = True

        for i in range(n):
            if i != j and not reach[i][j]:
                reachable_from_all = False
                break

        if reachable_from_all:
            return j

    return -1

graph = [
    [0, 1, 1],
    [0, 0, 1],
    [0, 0, 0]
]

print("Vertex:", find_reachable_vertex(graph))

graph = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

print("Vertex:", find_reachable_vertex(graph))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 12.py =====
Vertex: 2
Vertex: 0
>>>
```

13. Find the Vertex with Maximum Reachability

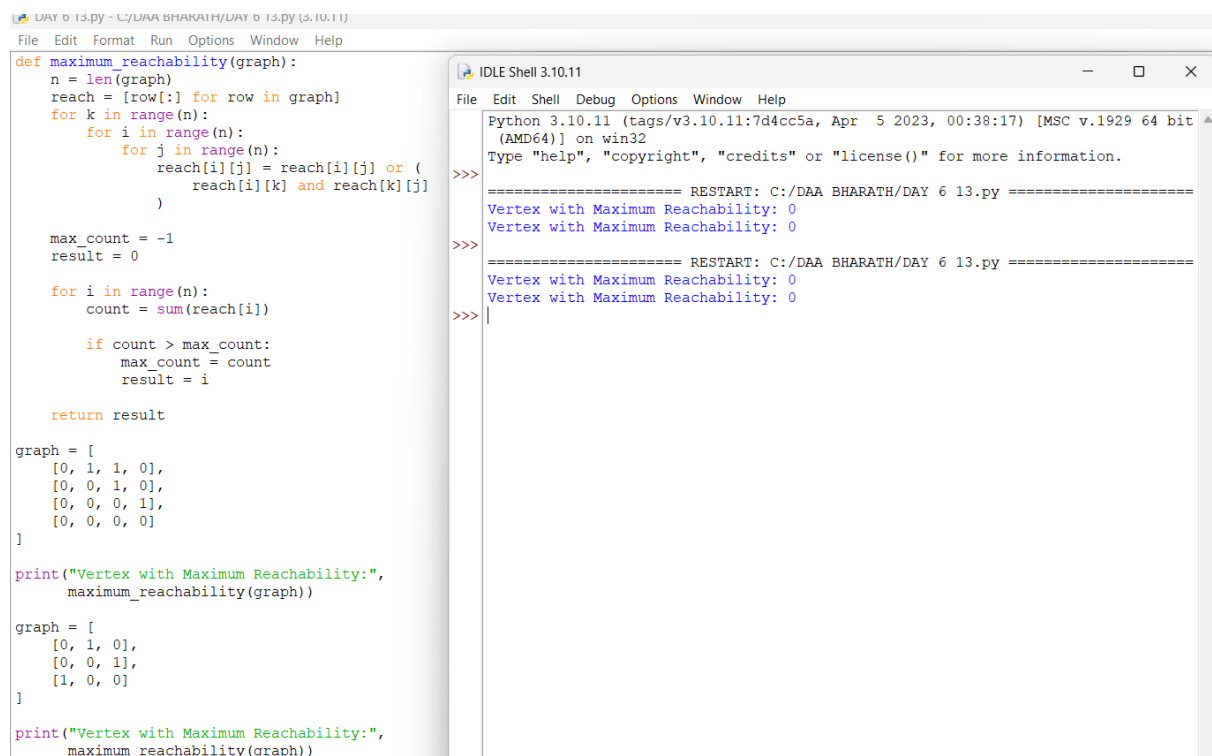
Aim

To find the vertex that can reach the maximum number of other vertices in a directed graph using Warshall's Algorithm. If there is a tie, return the smallest indexed vertex.

Algorithm

1. Start the program.
2. Initialize the reachability matrix with the given adjacency matrix.
3. Apply Warshall's Algorithm to find the transitive closure.
4. For each vertex, count the number of other vertices it can reach.
5. Compare the reachability counts of all vertices.
6. Select the vertex with the maximum count.
7. If there is a tie, select the smallest indexed vertex.
8. Display the result.
9. Stop the program

PROGRAM & OUTPUT



```
DAY 6 13.py - C:/DAA BHARATH/DAY 6 13.py (3.10.11)
File Edit Format Run Options Window Help

def maximum_reachability(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )

    max_count = -1
    result = 0

    for i in range(n):
        count = sum(reach[i])

        if count > max_count:
            max_count = count
            result = i

    return result

graph = [
    [0, 1, 1, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

print("Vertex with Maximum Reachability:",
      maximum_reachability(graph))

graph = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

print("Vertex with Maximum Reachability:",
      maximum_reachability(graph))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> ===== RESTART: C:/DAA BHARATH/DAY 6 13.py =====
Vertex with Maximum Reachability: 0
Vertex with Maximum Reachability: 0
>>> ===== RESTART: C:/DAA BHARATH/DAY 6 13.py =====
Vertex with Maximum Reachability: 0
Vertex with Maximum Reachability: 0
>>> |
```

14. Find the Diameter of a Weighted Graph

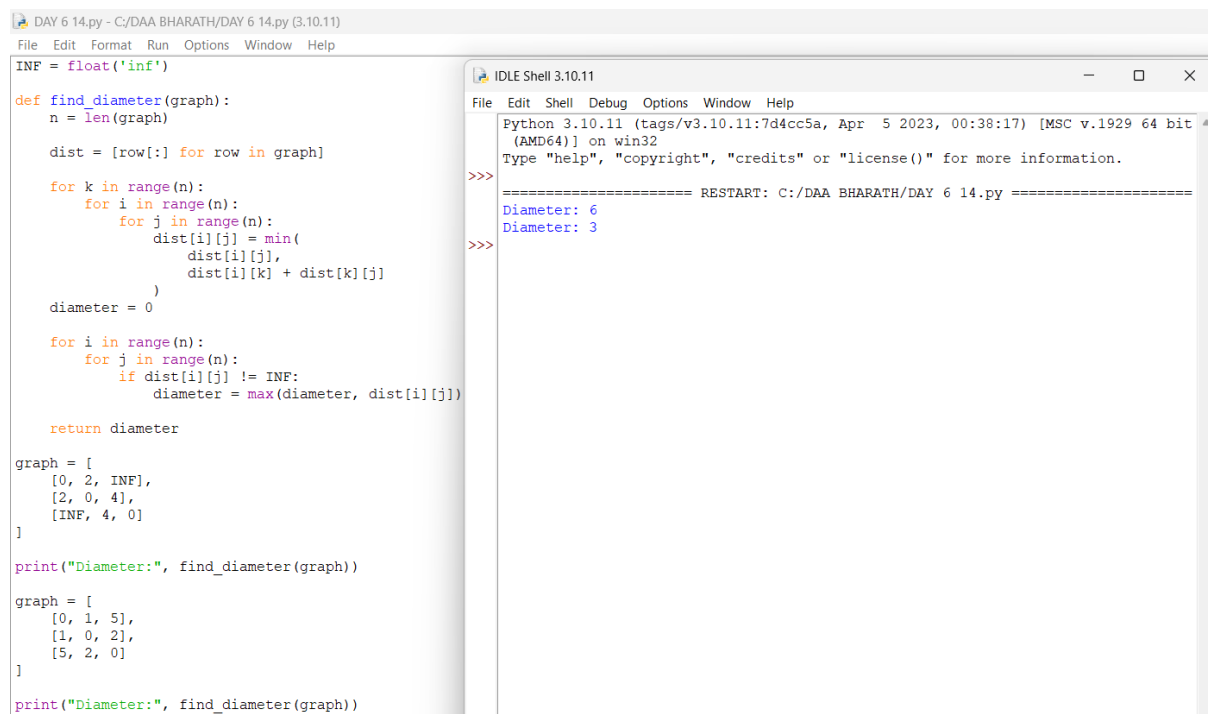
Aim

To find the diameter of a weighted graph, which is the maximum shortest-path distance between any pair of vertices, using the Floyd-Warshall Algorithm.

Algorithm

1. Start the program.
2. Initialize the distance matrix using the given weighted graph.
3. Consider each vertex as an intermediate vertex.
4. Apply Floyd-Warshall to find the shortest distance between every pair of vertices.
5. Find the maximum finite shortest-path distance.
6. Display the maximum distance as the diameter.
7. Stop the program.

PROGRFAM & OUTPUT



```
DAY 6 14.py - C:/DAA BHARATH/DAY 6 14.py (3.10.11)
File Edit Format Run Options Window Help

INF = float('inf')

def find_diameter(graph):
    n = len(graph)

    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    diameter = 0

    for i in range(n):
        for j in range(n):
            if dist[i][j] != INF:
                diameter = max(diameter, dist[i][j])

    return diameter

graph = [
    [0, 2, INF],
    [2, 0, 4],
    [INF, 4, 0]
]

print("Diameter:", find_diameter(graph))

graph = [
    [0, 1, 5],
    [1, 0, 2],
    [5, 2, 0]
]

print("Diameter:", find_diameter(graph))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 14.py =====
Diameter: 6
Diameter: 3
>>>
```

15. Check Whether a Graph is Strongly Connected

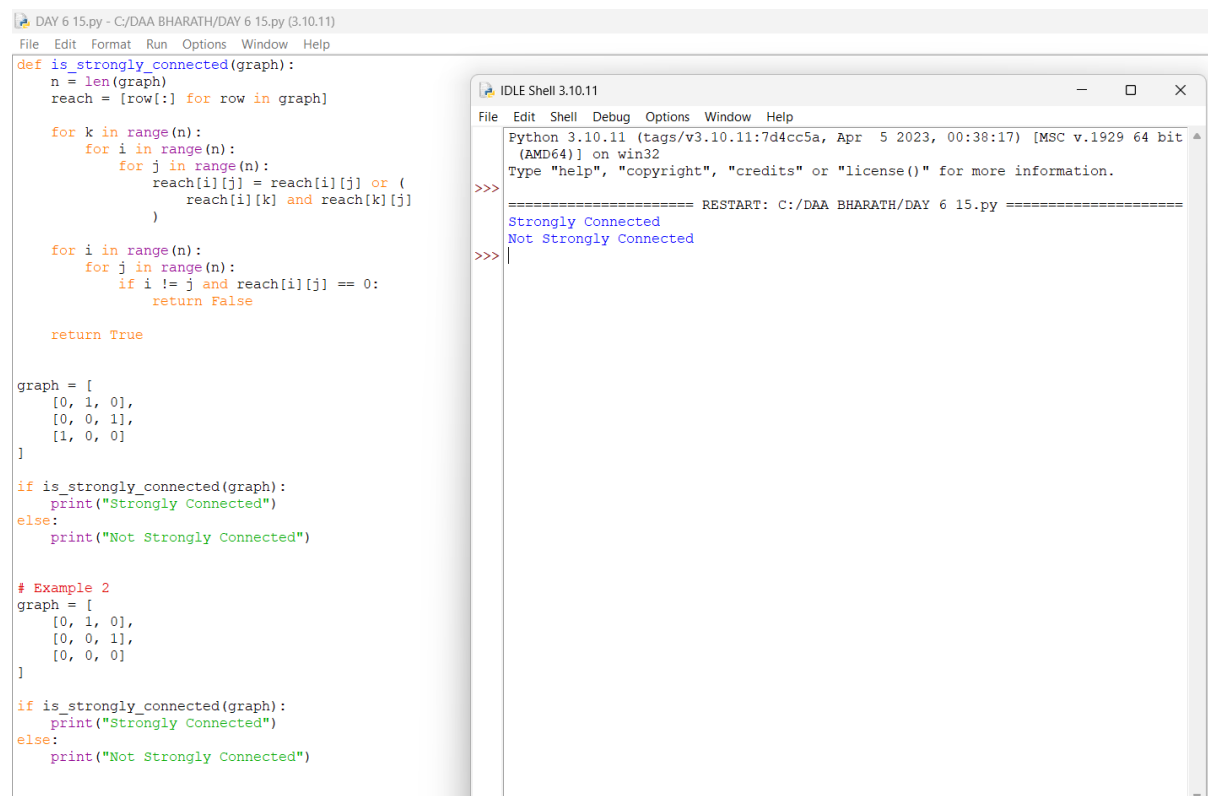
Aim

To determine whether every vertex in a directed graph is reachable from every other vertex using Warshall's Algorithm.

Algorithm

1. Start the program.
2. Initialize the reachability matrix with the adjacency matrix.
3. Apply Warshall's Algorithm to calculate the transitive closure.
4. Check whether every vertex can reach every other vertex.
5. If all pairs are reachable, print **Strongly Connected**.
6. Otherwise, print **Not Strongly Connected**.
7. Stop the program.

PROGRAM & OUTPUT



```
DAY 6 15.py - C:/DAA BHARATH/DAY 6 15.py (3.10.11)
File Edit Format Run Options Window Help

def is_strongly_connected(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (
                    reach[i][k] and reach[k][j]
                )

    for i in range(n):
        for j in range(n):
            if i != j and reach[i][j] == 0:
                return False

    return True

graph = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

if is_strongly_connected(graph):
    print("Strongly Connected")
else:
    print("Not Strongly Connected")

# Example 2
graph = [
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0]
]

if is_strongly_connected(graph):
    print("Strongly Connected")
else:
    print("Not Strongly Connected")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/DAA BHARATH/DAY 6 15.py =====
Strongly Connected
Not Strongly Connected
>>>|
```